Entregable Tecnico - Codebasicstor

1) Resumen del proyecto

Codebasicstor es una aplicacion web de reservas para servicios y productos, orientada a emprendimientos pequenos.
La arquitectura usa Next.js (App Router), Google Calendar como agenda central y panel admin con login Google.

Objetivo operativo:

- Mostrar servicios y productos.
- Permitir reservar horario.
- Registrar reserva pendiente.
- Aprobar desde panel admin.
- Enviar enlace de sesion por correo.

2) Stack tecnico

- Frontend: Next.js + React + CSS Modules.
- Backend: API Routes de Next.js (serverless-friendly).
- Auth admin: NextAuth con Google OAuth 2.0.
- Agenda: Google Calendar API.
- Correo: Nodemailer con Gmail App Password.
- Despliegue recomendado: Vercel.

3) Arquitectura funcional

3.1 Cliente final

- Home con servicios/productos.
- Flujo de reserva:

1. Datos del cliente.
2. Seleccion de fecha/hora segun disponibilidad.
3. Confirmacion de pago manual y solicitud.

3.2 Panel admin

- Acceso por Google Sign-In.
- Restriccion por correos autorizados (`ADMIN_EMAILS`).
- Vistas de:

1. Reservas pendientes.
2. Agenda confirmada.
3. Horario base (reglas de disponibilidad).

- Acciones clave:

1. Aprobar reserva.
2. Generar o pegar enlace Meet.
3. Enviar correo al cliente.

3.3 Capa API

Rutas principales:

- `/api/calendar/free-busy` : consulta disponibilidad diaria.
- `/api/calendar/create-pending` : crea reserva pendiente.

- `/api/admin/reservas` : lista reservas para admin.
- `/api/admin/approve-reserva` : aprueba reserva y actualiza evento.
- `/api/admin/update-meet` : agrega o genera enlace para sesion.
- `/api/admin/settings` : lee/guarda reglas de disponibilidad.
- `/api/cron/cleanup` : limpia reservas pendientes vencidas.

4) Fuente de verdad de agenda

Google Calendar es la fuente de verdad. La UI calcula bloques disponibles a partir de:

1. Horario base configurado en admin.
2. Bloqueos/ocupacion real del calendario.

Ventaja:

- Evita doble agenda cuando hay cambios fuera del sistema.
- Mantiene consistencia entre admin y calendario real.

5) Seguridad y control

- Login admin por Google OAuth.
- Lista blanca de admins con `ADMIN_EMAILS` .
- Secretos en variables de entorno, no en codigo.
- Cron protegido con `CRON_SECRET` .
- Rate limit en endpoints sensibles (ej. disponibilidad).

6) Variables de entorno (entregable)

Estas son las variables que el cliente final debe configurar.

Variable	Obligatoria	Uso	Ejemplo
`NEXTAUTH_URL`	Si	URL publica del sitio para auth	`https://tu-dominio.com`
`NEXTAUTH_SECRET`	Si	Firma de sesion NextAuth	`cadena_larga_segura`
`GOOGLE_CLIENT_ID`	Si	OAuth web client para login admin	`123...apps.googleusercontent.com`
`GOOGLE_CLIENT_SECRET`	Si	Secreto OAuth web client	`GOCSPX-...`
`ADMIN_EMAILS`	Si	Correos permitidos para entrar al admin	`admin@correo.com,otro@correo.com`
`GOOGLE_CLIENT_EMAIL`	Si	Email de service account para Calendar API	`reserva-bot@proyecto.iam.gserviceaccount.com`
`GOOGLE_PRIVATE_KEY`	Si	Llave privada del service account	`-----BEGIN PRIVATE KEY-----...`

`NEXT_PUBLIC_CALENDAR_ID`	Si	ID del calendario objetivo	`owner@gmail.com` o `...@group.calendar.google.com`
`GMAIL_USER`	Si	Cuenta remitente de correos	`notificaciones@dominio.com`
`GMAIL_APP_PASSWORD`	Si	App password de Gmail (no password normal)	`abcd efgh ijkl mnop`
`CRON_SECRET`	Si en prod	Token bearer para cron cleanup	`token_largo_aleatorio`

7) Como cambiar el admin

Cambiar admins no requiere tocar codigo.

Pasos:

1. Editar `ADMIN_EMAILS` con uno o varios correos separados por coma.
2. Redeploy.
3. Probar login en `/admin/reservas`.

8) Parametros de marca que suelen personalizarse

- Nombre del sitio en metadata.
- Nombre visible del remitente de email (hoy esta en `src/lib/mailer.ts`).
- Catalogo de servicios/productos (`src/lib/data.ts`).
- Numeros/cuentas de pago manual en pantalla.

9) Estado de esta version entregable

- Sin integracion activa de pasarela de pagos.
- Sin dependencias de Calendly.
- Flujo principal centrado en:

1. Reserva.
2. Aprobacion admin.
3. Envio de enlace y correo.